

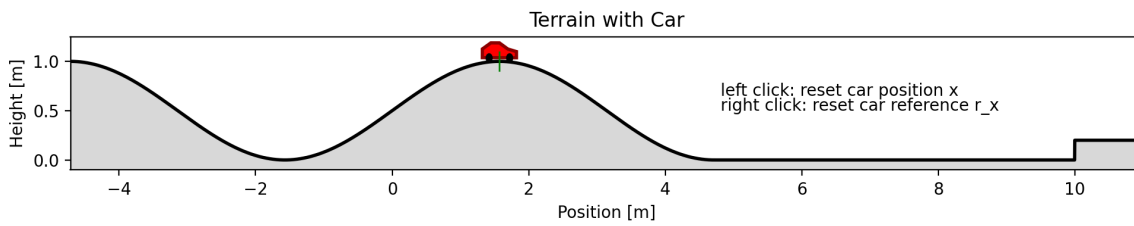
Prof. Dr. Stefan Leutenegger

Teaching Assistants: David Rohr, Alexander Hansson, Leonard Freissmuth, Hanzhi Chen

Solutions to Exercise 10: PID & MPC

Session date: 12/05/2026

Consider a car moving over hills as illustrated in the figure below. In this exercise, you will implement two controllers, a Proportional-Integral-Derivative (PID) controller and a Model-Predictive Controller (MPC) to control the car's position, e.g. to stabilize it on top of a hill, or to stop closely in front of the wall on the right without crashing.



The hills here are very specific and their height profile is

$$h(x) = \begin{cases} a \sin(x) & -\frac{3\pi}{2} \leq x \leq \frac{3\pi}{2} \\ 0 & \frac{3\pi}{2} < x \leq 10 \\ 0.25 & x > 10 \end{cases} \quad a > 0.$$

The car is of mass m and can be controlled with a forward facing force $u \in [F_{\min}, F_{\max}]$. Gravity further accelerates or decelerates the car as $-mg \sin(\alpha)$ tangentially to the curve, where $\alpha = \arctan\left(\frac{dh(x)}{dx}\right)$ denotes the local inclination. Apart from this and the gravity component in the up/down parts, the car also experiences friction that is modelled as $F_d = k_1 v + k_2 v|v|$, where $k_1 > 0$ and $k_2 > 0$. With this, we can approximate the car's dynamics:

$$\begin{aligned} \dot{x} &= v \cos \alpha(x) \\ \dot{v} &= -g \sin \alpha(x) - \frac{1}{m}(k_1 v + k_2 v|v|) + \frac{1}{m}u \end{aligned}$$

where x stands for the position along the horizontal axis and v for the velocity (tangential to the curve). For the flat part, you can simply set $\alpha(x) = 0$, for the hilly part we can expand the expressions to

$$\begin{aligned} \dot{x} &= \frac{v}{\sqrt{a^2 \cos^2 x + 1}}, \\ \dot{v} &= \frac{-ag \cos x}{\sqrt{a^2 \cos^2 x + 1}} - \frac{1}{m}(k_1 v + k_2 v|v|) + \frac{1}{m}u. \end{aligned}$$

Task 1: PID Position Control of Car Driving Over Hills

- i) Determine the equilibria of the so-called “autonomous” ($u = 0$), open-loop system, i.e. the states where $\dot{\mathbf{x}} = [\dot{x}, \dot{v}]^T = \mathbf{0}$? Perform a linear stability analysis to classify these equilibria as stable or unstable.

Hint:

$$\frac{d}{dx} \left(\frac{1}{\sqrt{\cos^2 x + 1}} \right) = \frac{\cos x \sin x}{(\cos^2 x + 1)^{1.5}}$$

$$\begin{aligned}\dot{x} = 0 &= \frac{v}{\sqrt{a^2 \cos^2 x + 1}}, \\ \dot{v} = 0 &= \frac{-ag \cos x}{\sqrt{a^2 \cos^2 x + 1}} - \frac{1}{m}(k_1 v + k_2 v|v|),\end{aligned}$$

The first equation can only be fulfilled for $v = 0$. Plugging this into the second equation, we must fulfil $-ag \cos(x) = 0$, hence $\cos(x) = 0$. In the relevant hilly interval $x \in [-3\pi/2, 3\pi/2]$, this yields three equilibria: $x_1 = -3\pi/2$, $x_2 = -\pi/2$, and $x_3 = \pi/2$. The points $x_1 = -3\pi/2$ and $x_3 = \pi/2$ are hill crests (h), while $x_2 = -\pi/2$ is a valley (v). Using linearization, we find the system matrix $\mathbf{F}_c$ to be, for these two cases, respectively:

$$\begin{aligned}\mathbf{F}_{c,h} &= \begin{bmatrix} 0 & 1 \\ ag & -k_1/m \end{bmatrix}, \\ \mathbf{F}_{c,v} &= \begin{bmatrix} 0 & 1 \\ -ag & -k_1/m \end{bmatrix}.\end{aligned}$$

To compute its eigenvalues, we need to solve the characteristic polynomial $\det(\mathbf{F}_c - \lambda \mathbf{I}) = 0$:

$$\begin{aligned}0 &= -\lambda_h(-k_1/m - \lambda_h) - ag, \\ 0 &= -\lambda_v(-k_1/m - \lambda_v) + ag.\end{aligned}$$

Solving these quadratic equations, we get

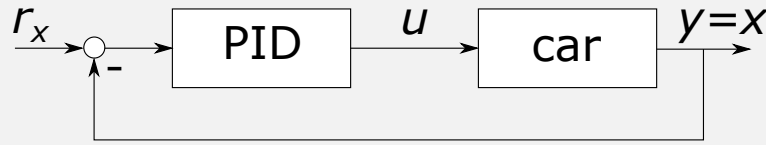
$$\begin{aligned}\lambda_{h,1,2} &= \frac{-k_1 \pm \sqrt{k_1^2 + 4agm^2}}{2m}, \\ \lambda_{v,1,2} &= \frac{-k_1 \pm \sqrt{k_1^2 - 4agm^2}}{2m}.\end{aligned}$$

We remember that $m > 0, a > 0, k_1 > 0, g = 9.81\text{m/s}^2$. So, for the hill-case, we have in fact two real eigenvalues, whereby one of them $\lambda_{h,2} > 0$, i.e. the hill is unstable (which we would of course have guessed). Note that since the two values are real, there is no oscillatory behaviour: this can be perhaps intuitively explained by the fact that we were using the linearization as a local approximation, therefore the system behaves as if there were exactly one hill at the selected linearization point...

As for the interior valley at $x = -\pi/2$, we realize that for a say “low-friction”-regime with $k_1^2 < 4agm^2$, we get a complex pair $\lambda_{v,1,2}$ with negative real part; this means stable oscillatory behavior. Again, we would have probably expected this for the valley: the car will move back-and-forth around the lowest point, until, due to friction, it comes to a stand-still. In the “high-friction”-regime, however, both eigenvalues $\lambda_{v,1,2}$ are negative real numbers, therefore this valley is a stable equilibrium, where no oscillations occur.

- ii) Draw a block diagram of the PID position loop using x as the measured output. Implement the law directly in `E10_hill_driving.py` (e.g. in `PidController.control`), using the setpoint r_x , measured position x , and the controller step interval already defined in the simulator (e.g. $\Delta t = 1/\text{FPS}$). Start with $k_i = 0$, then tune k_p and k_d until the closed-loop behavior is satisfactory. Briefly discuss: can you regulate the car to a constant position on the slope? Can you come to rest near the far wall without overshooting? In what situations does integral action help, and what are its drawbacks here?

We can use the very standard PID signal diagram as well as implementation and assume we can measure the position x as a system output, i.e. $y = x$:



We observe the following things:

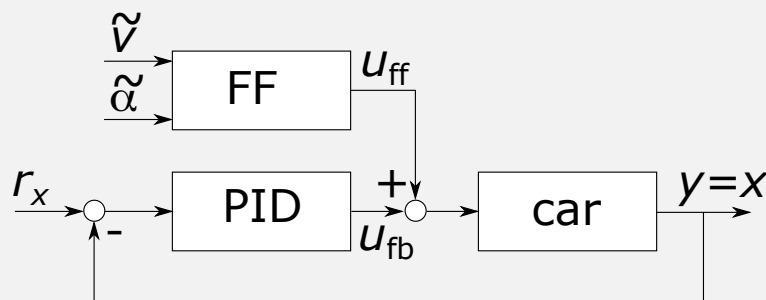
- With a pure PD controller a steady-state offset is common because gravity and friction act as constant disturbances.
- Stopping near the far wall without overshoot is possible but requires conservative tuning: high k_p and low damping (k_d too small) tends to produce oscillation and overshoot, while stronger damping improves safety but slows response.
- Integral action ($k_i \neq 0$) helps when persistent bias is present: it drives the remaining steady-state position error to zero.
- Main drawbacks of integral action here are slower transients and windup under actuator saturation, which can increase overshoot unless anti-windup is used.

- iii) Sketch a modified architecture that adds a feed-forward path aimed at keeping v approximately constant along the slope (this is a relaxed notion of feed-forward: you may use a speed measurement $\tilde{v}$ and an estimate of the local inclination $\tilde{\alpha}$). Give the feed-forward force u_{ff} and implement it in `E10_hill_driving.py` alongside the PID loop. In a few bullet points, explain why you would add this additional component.

We can compensate the downward force and friction with the feed-forward component u_{ff} . This can be derived by setting $\dot{v} = 0$, i.e.

$$\begin{aligned} \dot{v} = 0 &= -g \sin(\tilde{\alpha}) - \frac{1}{m}(k_1 \tilde{v} + k_2 \tilde{v}|\tilde{v}|) + \frac{1}{m}u_{ff}, \\ u_{ff} &= mg \sin(\tilde{\alpha}) + (k_1 \tilde{v} + k_2 \tilde{v}|\tilde{v}|). \end{aligned}$$

The modified control structure looks now as follows:



This scheme can give us better overall control performance, because:

- We have removed parts of the nonlinear effects and compensated them with the term above.
- Therefore, the remaining system – that still needs feedback control to drive the position error to zero – will require less control effort, as it can be viewed as

$$\begin{aligned}\dot{x} &= \frac{v}{\sqrt{a^2 \cos^2 x + 1}}, \\ \dot{v} &= \frac{1}{m} u_{fb},\end{aligned}$$

which is much closer to a linear system (in fact the sub-system for v has become linear – under the assumption that the compensation is “perfect”) where we know PID works really well.

Task 2: MPC Position Control of Car Driving Over Hills

Instead of feeding back with a handcrafted law, we now want to use linear MPC to control the car’s position (full state reference $\mathbf{r}$), employing linearization again, but this time around the current x and $\bar{v} = 0$, i.e. $\bar{\mathbf{x}} = [x, 0]^T$. In addition, we implicitly choose the linearization point $\bar{u}$ such that we have $\dot{v}(\bar{x}) = 0$. To move to discrete time, we employ Euler-forward for simplicity, so by application of the chain rule, the matrices $\mathbf{F}$ and $\mathbf{G}$ of the linearized discrete time system (assuming a constant time increment Δt) simply become $\mathbf{F} = \mathbf{I} + \Delta t \mathbf{F}_c$ and $\mathbf{G} = \Delta t \mathbf{G}_c$.

Now consider the finite-horizon quadratic cost

$$\begin{aligned}J(\delta \mathbf{x}_0, \dots, \delta \mathbf{x}_N, \delta u_0, \dots, \delta u_{N-1}) &= (\bar{\mathbf{x}} + \delta \mathbf{x}_N - \mathbf{r})^T \mathbf{P} (\bar{\mathbf{x}} + \delta \mathbf{x}_N - \mathbf{r}) \\ &+ \sum_{k=0}^{N-1} [(\bar{\mathbf{x}} + \delta \mathbf{x}_k - \mathbf{r})^T \mathbf{Q} (\bar{\mathbf{x}} + \delta \mathbf{x}_k - \mathbf{r}) + R(\bar{u} + \delta u_k)^2]\end{aligned}$$

and the control optimization problem

$$\begin{aligned}J^* &= \min_{\delta u_0, \dots, \delta u_{N-1}} J(\delta \mathbf{x}_0, \dots, \delta \mathbf{x}_N, \delta u_0, \dots, \delta u_{N-1}), \\ \text{s.t.} \quad \delta \mathbf{x}_{k+1} &= \mathbf{F} \delta \mathbf{x}_k + \mathbf{G} \delta u_k, \\ \bar{x} + \delta x_k &\leq 10, \quad k = 0, \dots, N-1, \\ \bar{u} + \delta u_k &\geq u_{\min}, \quad k = 0, \dots, N-1, \\ \bar{u} + \delta u_k &\leq u_{\max}, \quad k = 0, \dots, N-1, \\ \bar{\mathbf{x}} + \delta \mathbf{x}_0 &= \mathbf{x}.\end{aligned}$$

Assume a time discretization of $\Delta t = 0.05$ seconds, and a horizon of $N = 100$ (i.e. 5 s).

- Implement the missing parts of the MPC controller `MpcController.control` in `E10_hill_driving.py`. Use `qp solvers` to solve the QP online as stated above.

See `E10_hill_driving_sol.py`.

- Tune $\mathbf{P}$, $\mathbf{Q}$, and R (choose diagonal matrices) and observe the behavior for different reference positions.

To compute the linearization point for u , we get for the hilly part

$$\dot{v}_{\text{hill}} = \frac{-ag \cos \bar{x}}{\sqrt{a^2 \cos^2 \bar{x} + 1}} + \frac{1}{m} \bar{u} \stackrel{!}{=} 0 \rightarrow \bar{u}_{\text{hill}} = \frac{amg \cos \bar{x}}{\sqrt{a^2 \cos^2 \bar{x} + 1}}$$

and for the flat section

$$\dot{v}_{\text{flat}} = \frac{1}{m} \bar{u} \stackrel{!}{=} 0 \rightarrow \bar{u}_{\text{flat}} = 0.$$

With this, we get for the matrices $\mathbf{F}_c$ and $\mathbf{G}_c$

$$\begin{aligned} \mathbf{F}_{c,\text{flat}} &= \begin{bmatrix} 0 & 1 \\ 0 & -k_1/m \end{bmatrix}, \\ \mathbf{F}_{c,\text{hill}} &= \begin{bmatrix} 0 & \frac{1}{\sqrt{a^2 \cos^2(\bar{x}) + 1}} \\ \frac{ag \sin(\bar{x})}{(a^2 \cos^2(\bar{x}) + 1)^{\frac{3}{2}}} & -k_1/m \end{bmatrix}, \\ \mathbf{G}_c &= \begin{bmatrix} 0 \\ 1/m \end{bmatrix}. \end{aligned}$$

Note the solutions in the code contain the more general equations for linearizing around non-zero $\bar{v}$. Next, we need to take the above into the form required by `qp solvers`, which is

$$\begin{aligned} J^* &= \min_{\mathbf{x}_q} \quad \frac{1}{2} \mathbf{x}_q^T \mathbf{P}_q \mathbf{x}_q + \mathbf{q}_q^T \mathbf{x}_q, \\ \text{s.t.} \quad \mathbf{G}_q \mathbf{x}_q &\leq \mathbf{h}_q, \\ \mathbf{A}_q \mathbf{x}_q &= \mathbf{b}_q. \end{aligned}$$

Hereby, we realize that all optimization variables $\mathbf{x}_q$ need to include both the state sequence as well as the input sequence, i.e.

$$\mathbf{x}_q = [\delta \mathbf{x}_0^T, \dots, \delta \mathbf{x}_N^T, \delta u_0, \dots, \delta u_{N-1}]^T.$$

With this, we find

$$\begin{aligned} \mathbf{P}_q &= 2 \text{blkdiag}(\mathbf{Q}, \dots, \mathbf{Q}, \mathbf{P}, R, \dots, R) \\ \mathbf{q}_q &= 2 [(\mathbf{Q}(\bar{\mathbf{x}} - \mathbf{r}))^T, \dots, (\mathbf{Q}(\bar{\mathbf{x}} - \mathbf{r}))^T, (\mathbf{P}(\bar{\mathbf{x}} - \mathbf{r}))^T, R\bar{u}, \dots, R\bar{u}]^T, \\ \mathbf{G}_q &= \begin{bmatrix} 0 & \dots & 0 & 0 & 1 & \dots & 0 \\ 0 & \dots & 0 & 0 & -1 & \dots & 0 \\ \vdots & & \vdots & \vdots & & \ddots & \\ 0 & \dots & 0 & 0 & 0 & \dots & 1 \\ 0 & \dots & 0 & 0 & 0 & \dots & -1 \end{bmatrix}, \\ \mathbf{h}_q &= [u_{\max} - \bar{u}, -u_{\min} + \bar{u}, \dots, u_{\max} - \bar{u}, -u_{\min} + \bar{u}]^T, \\ \mathbf{A}_q &= \begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{F} & -\mathbf{I} & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{G} & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{F} & -\mathbf{I} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{G} & \dots & \mathbf{0} \\ \vdots & & \ddots & \ddots & & \vdots & \vdots & & \ddots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots & \mathbf{F} & -\mathbf{I} & \mathbf{0} & \mathbf{0} & \dots & \mathbf{G} \end{bmatrix}, \\ \mathbf{b}_q &= [0, v, \mathbf{0}]^T. \end{aligned}$$

If you don't believe it, multiply it out (block-wise)...

iii) Briefly discuss: can you regulate to a constant position on the slope? Can you come to rest near the far

wall without overshooting? What are the advantages and disadvantages of MPC compared to PID?

- Yes, with suitable weights, horizon length and an accurate model, MPC can regulate to a constant position on the slope with a near-zero steady state error.
- Resting near the far wall without overshoot is typically easier with MPC, because input limits are handled explicitly in the optimization and future motion is anticipated over the horizon.
- Compared to PID, MPC is more systematic for multi-objective tuning (tracking vs. actuation effort) and naturally incorporates constraints.
- Main disadvantages are higher computational cost, dependence on model quality and sensitivity to QP failures/poor conditioning.